

Cuvântul `static`

Teorie minimală + șapte exerciții, pe codul tău

Teza fișei: ce e `static` aparține **clasei**, nu unui obiect. Există într-un singur exemplar, oricâte obiecte ai face, chiar și zero.

1. Îl folosești deja, de cel puțin șase ori

Tot ce chemi pe numele unei clase, nu pe un obiect

```
public static void main(String[] args)    // App
public static void exOrar()               // App
Math.PI, Math.abs(...)                   // Cerc, Dreptunghi
Collections.sort(this.arrIntervale)       // Orar
Integer.compare(a, b)                    // Interval
Arrays.asList(curs1, pauza1, ...)         // App
```

N-ai scris niciodată `new Math()` ca să afli cât e π . Valoarea lui π nu depinde de niciun obiect, deci stă direct pe clasă. La fel `main`: Java îl pornește **înainte** să existe vreun obiect, deci trebuie să poată fi chemat fără unul.

2. Câmp de instanță vs câmp `static`

Fiecare obiect are câmpul lui, sau există unul singur pentru toți

```
class Pauza1 {                          class Pauza2 {
    int create = 0;                      static int create = 0;
    Pauza1() { create++; }              Pauza2() { create++; }
}
```

Faci trei obiecte din fiecare și afișezi contorul:

```
Pauza1: a.create=1 b.create=1 c.create=1
Pauza2: Pauza2.create=3
```

În `Pauza1` fiecare obiect se naște cu propriul `create = 0` și îl crește doar pe al lui. În `Pauza2` câmpul există o singură dată, pe clasă, iar toți constructorii îl cresc pe același. De-aia se scrie `Pauza2.create`, cu numele clasei.

	câmp de instanță	câmp <code>static</code>
câte exemplare	unul pentru fiecare obiect	unul singur, pe clasă
se citește cu	<code>obiect.camp</code>	<code>Clasa.camp</code>
exemplu la tine	<code>oraInceput</code> , <code>raza</code>	un contor, o constantă

3. Metoda `static` n-are `this`

O metodă statică se cheamă pe clasă, deci nu există niciun obiect „al ei”. Fără obiect nu are nici `this`, nici câmpurile lui. De-aia, dacă din `main` încerci să citești un câmp al obiectului, compilatorul te oprește:

```
error: non-static variable oraInceput cannot be referenced
      from a static context
```

Mesajul nu înseamnă „pune `static` peste tot”. Înseamnă că întrebarea „al cui `oraInceput` ?” nu are răspuns. Invers merge oricând: o metodă de instanță poate chema una statică.

4. Constante: `static final`

În `Interval` numărul `60` apare de peste zece ori, iar `1440` o dată. Un cititor nu știe dacă fiecare `60` înseamnă „minute într-o oră” sau „o durată de 60 de minute”, adică exact confuzia din bug-urile de decalare.

```
static final int MINUTE_PE_ORA = 60;
```

`static`: e aceeași valoare pentru toate intervalele. `final`: nu se mai poate schimba. Numele se scrie cu majuscule și `_`.

5. Capcana: stare comună fără să vrei

Un câmp `static` care se **modifică** e o variabilă comună tuturor obiectelor. Dacă `arrIntervale` din `Orar` ar fi `static`, toate orarele ar avea aceeași listă: adaugi la luni, apare și la marți. Contorul e una dintre puținele excepții în care asta e chiar ce vrei.

Cum decizi: metoda folosește ceva din obiect (un câmp, `this`)? Atunci **nu** e statică. Nu folosește nimic din obiect? E candidată la `static`. Semnul clar: dacă scrii `new X()` doar ca să poți chema o metodă, metoda trebuia să fie statică.

Exerciții

Unde lucrezi: repo-ul `java-abstractizare`, peste orar și figuri.

Nou, ai voie: câmpuri și metode `static`, `static final`, constructor `private`.

Încă NU folosim: metode `static` în interfețe, `enum`, `record`, `try/catch`, `stream-uri`.

Cum se lucrează: fiecare exercițiu îți cere să **construiești** ceva. Rândul **Gata când** spune cum verifici: prin rulare sau cu `grep`.

1. Numerele fără nume din `Interval`

nivel 1

Înlocuiește fiecare `60` și `1440` din `Interval` cu o constantă `static final` cu nume care spune ce înseamnă.

Gata când: `grep -n "60\|1440" src/app/OrarScheduler/Interval.java` găsește numerele doar pe liniile unde declari constantele, iar programul afișează exact ce afișa înainte.

2. Fiecare interval primește un număr

nivel 1

Fiecare interval nou primește automat un `id`: primul creat are 1, al doilea 2 și tot așa. Id-ul **nu** se dă din constructor, de afară. Copia făcută cu `duplicate()` e un interval nou, deci primește și ea un id nou. Adaugă și o metodă pe clasă care spune câte intervale s-au creat până acum.

Gata când: creezi 5 intervale și duplici unul, iar afișarea arată id-urile 1–6, toate diferite. Metoda de numărare, chemată pe **numele clasei**, dă 6.

De reținut: id-ul e al fiecărui obiect, contorul e al clasei. Ai nevoie de câte un câmp pentru fiecare.

3. `CrossClass` fără obiect

nivel 1

`CrossClass` nu are niciun câmp, iar în `App` faci `new CrossClass()` doar ca să-i chemi metoda de copiere. Fă metoda statică și cheam-o pe numele clasei.

Gata când: `grep -rn "new CrossClass" src` nu găsește nimic, iar lista de copii se afișează la fel ca înainte.

4. Constructori cu nume

nivel 2

Scrie `Punct.origine()`, care întoarce punctul (0,0), și două metode statice pe `Pauza`: una pentru pauza scurtă de 10 minute, una pentru pauza mare de 20, amândouă primind doar ora. Folosește-le în `App` în locul lui `new`.

Gata când: muți un punct luat cu `Punct.origine()`, apoi chemi din nou `Punct.origine()`, iar al doilea punct e tot `X:0,Y:0`.

De reținut: un singur `Punct` static, întors de fiecare dată, ar fi stare comună (teoria, secțiunea 5).

5. Clasa `Timp`

nivel 2

Toată aritmetica de minute din `Interval` se mută într-o clasă `Timp` cu metode statice: din oră și minut în minute, și din minute în textul `hh:mm`. `Timp` nu are stare, deci nimeni nu trebuie să poată face `new Timp().afisare()`, `toString()` și `decalare()` o folosesc.

Gata când: `new Timp()` nu compilează, `Interval` nu mai conține nici `/ 60`, nici `% 60`, iar un curs de la 10:00 afișează 10:00, oricât ar dura.

De reținut: asta încheie B5 din review: formatul `hh:mm` există într-un singur loc.

6. Un curs care începe la 8:30

provocare

Adaugă în `Timp` o metodă care transformă textul `"8:30"` în minute. Apoi fă ca un interval să poată începe la o oră care nu e fixă, construit din text.

Gata când: un curs creat din `"8:30"` afișează 8:30, iar după `decalare(45)` afișează 9:15.

De reținut: nu merge fără să închezi întâi B4 din review. Câmpul trebuie să țină minute de la un capăt la altul.

7. Orarul refuză suprapunerile

provocare

`adaugaInterval` nu mai acceptă un interval care se suprapune cu unul deja existent și afișează un mesaj. Verificarea „se suprapun două intervale?” lucrează doar cu numere (început și durată pentru fiecare), deci e o metodă statică în `Timp`.

Gata când: după un curs 10:00–11:00, un examen la 10:30 e refuzat, o pauză la 11:00 e acceptată (capetele care se ating nu se suprapun), iar orarul afișează doar ce a fost acceptat.